과제 제출물

바이브코딩으로 만드는 교육용 앱 개발

과제 보고서(상세본) — 5개 교육용 앱 개발 결과

과제명	바이브코딩으로 만드는 교육용 앱 개발
강좌	2026학년도 2학기 AI 활용 교육 강좌
제출자	곽창섭
제출일	2026-10-28
개발 도구	Claude Code(바이브코딩), HTML/CSS/JavaScript
구성	상세본(본 문서) · 요약본(별도 파일)

내장된 학교·학급·학생 데이터는 모두 가상의 샘플입니다.

목차

1. 과제 개요 3
2. 개발한 앱 요약 4
2.1 분수교구함 — 01. 교과 교육지원 앱 4
2.2 기억상자 — 02. 학습지도용 앱 6
2.3 자리뚝딱 — 03. 학교 업무 자동화 앱 7
2.4 특별실지기 — 04. 학교 행정 지원 앱 8
2.5 연산수호대 — 05. 교육용 게임 앱 9
3. 공통 설계와 기술 구조 10
4. 바이브코딩 개발 과정 11
4.1 전체 절차와 각 단계의 지시 요지 12
4.2 멀티 에이전트 워크플로 13
4.3 자동화 검증 도구 14
4.4 반복 횟수와 결함 수 15
4.5 프롬프트 작성에서 배운 점 15
5. 품질 검증 결과 16
6. 활용 방안과 기대 효과 18
7. 한계와 향후 과제 19
8. 소감 20
부록 A. 제출물 목록과 폴더 구조 21
부록 B. 실행 방법 21
부록 C. 온라인 실행 링크 22

1. 과제 개요

이 보고서는 과제 「바이브코딩으로 만드는 교육용 앱 개발」의 수행 결과를 전체적으로 조망하는 문서이다. 과제가 요구한 5개 카테고리마다 웹앱 1개씩, 총 5개의 교육용 앱을 AI 코딩 에이전트(Claude Code)와 자연어로 대화하며 개발했다. 앱별 상세 내용은 각 앱의 「개요 및 설명서」로 별도 제출하고 핵심만 간추린 「요약본」도 함께 제출하므로, 상세본인 이 문서는 요구사항과 접근 방식, 5개 앱 요약, 공통 설계, 개발 과정, 품질 검증 결과, 활용 방안과 한계, 소감을 다룬다.

과제 요구사항. ① 교과 교육지원 앱(특정 교과·단원 수업에서 교사의 설명과 학생의 탐구를 직접 돕는 도구), ② 학습지도용 앱(교과를 가리지 않고 학습 방법·습관·자기 점검을 지원하는 도구), ③ 학교 업무 자동화 앱(담임·교과 교사가 반복 수행하는 학급 운영 업무를 대신 처리하는 도구), ④ 학교 행정 지원 앱(학교 단위의 운영·관리 업무를 돕는 도구), ⑤ 교육용 게임 앱(게임 메커니즘과 학습 내용이 맞물린 놀이형 학습 도구)의 다섯 카테고리에 각각 하나의 앱을 개발하고, 앱마다 개요·설명서 문서와 소스 압축 파일, 실제로 실행되는 구현 페이지를 제출한다. 개발 기간은 2026-10-17 ~ 2026-10-25(세션 중단 기간 포함)이고, 2026-10-26에 사용자 피드백에 따라 분수교구함을 단일 화면으로 단순화해 다시 구현했으며, 2026-10-28에 다섯 앱 전체 재점검·오류 수정과 보고서 작성·패키징을 마쳤다.

목표. (1) 다섯 카테고리의 경계가 겹치지 않도록 앱의 역할을 분명히 나누고 각 앱이 해당 카테고리의 대표 사례가 되도록 설계한다. (2) 학교 PC·태블릿·전자칠판 어디서나 설치·서버·로그인 없이 `index.html` 더블클릭으로 실행되는 완성품을 만든다. (3) AI가 작성한 코드를 그대로 믿지 않고, 만든 주체와 검증하는 주체를 분리한 자동화 검증으로 결함을 찾아 수정하는 과정을 끝까지 수행한다. (4) AI와의 협업 과정(프롬프트·절차·수치)을 사실대로 기록한다.

전체 접근 방식 — 바이브코딩. 바이브코딩(vibe coding)은 개발자가 코드를 직접 타이핑하는 대신 자연어로 요구사항·제약·검증 기준을 AI 코딩 에이전트에게 전달하고, AI가 설계·구현·테스트·문서 초안을 수행하며, 사람은 결과를 검토하고 방향을 결정하는 개발 방식이다. 이 과제에서 개발자(제출자)는 **결정**(기술 제약, 카테고리 해석, 공통 규약 확정, 심사 결과 승인), **지시**(단계마다 역할·목표·금지 사항·완료 조건을 문장으로 명시한 프롬프트 작성), **검토**(스펙·스크린샷·검증 보고서를 읽고 다음 라운드 여부 판단, 중간 산출물을 파일과 Git 커밋으로 보존)를 맡았다.

> 핵심 원칙은 「만드는 에이전트와 검증하는 에이전트를 분리한다」였다. 빌더가 「수용 기준 전부 통과」라고 보고해도 소스를 수정할 수 없는 독립 검증 에이전트 3명(기능 테스트·코드 리뷰·UX 리뷰)이 다시 확인하고, 수정 뒤에는 또 다른 재검증 에이전트가 수정 주장을 믿지 않고 직접 재현하는 구조를 모든 앱에 동일하게 적용했다.

2. 개발한 앱 요약

다섯 앱은 모두 가상의 「가온초등학교 5학년 2반(담임 이가람, 학생 24명)」 샘플 데이터를 공유하되, 카테고리마다 서로 다른 대표 기술을 하나씩 갖도록 설계했다. 각 앱은 독립 폴더에 들어 있으며 공통 파일을 참조하지 않는다.

번호	카테고리	앱 이름	한 줄 소개	대표 기술	폴더명
1	교과 교육 지원	분수교구함 (fraction-kit)	분수 하나를 막대·원·수직선으로 함께 보는 단일 화면 분수 교구	SVG 3표상 실시간 동기화 + 분할선 애니메이션	01_교과교육지원_분수교구함
2	학습지도	기억상자 (memory-box)	라이트너 상자 방식의 간격 반복 복습·오답 노트	간격 반복 스케줄러 + 직접 그린 SVG 차트	02_학습지도_기억상자
3	학교 업무 자동화	자리뚝딱 (jari-ttukttak)	조건만 체크하면 자리표·모둠·역할을 자동 생성하는 담임 도우미	제약 만족 배치 엔진 + Pointer Events 드래그·반전 인쇄	03_업무자동화_자리뚝딱
4	학교 행정 지원	특별실지기 (special-room-booking)	특별실·기자재 주간 예약판, 이중 예약 자동 검사와 실적 보고	교시 격자 드래그 예약·충돌 검사 + 형평성 히트맵	04_행정지원_특별실지기
5	교육용 게임	연산수호대 (math-guardians)	답을 입력해 쏘는 연산 타워 디펜스, 틀린 문제는 복수 몬스터로	Canvas 게임 루프 + Web Audio 합성 효과음	05_교육용게임_연산수호대

2.1 분수교구함 — 01. 교과 교육지원 앱

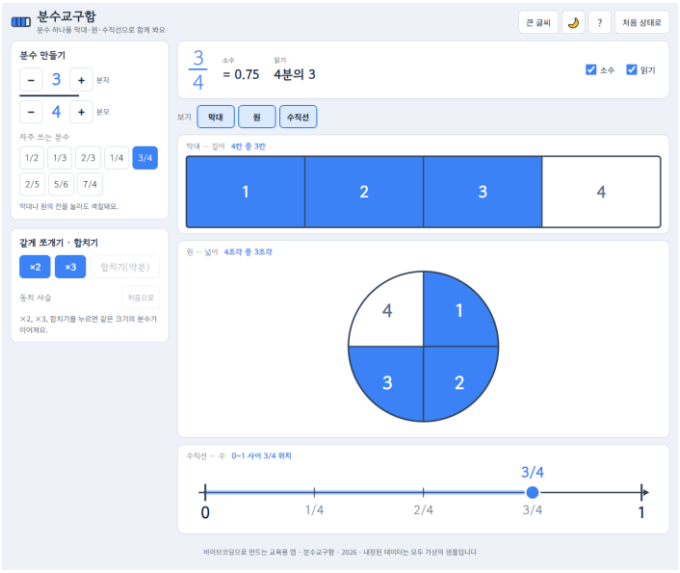


그림 1. 분수교구함(단순화판) 교구판 — 분수 만들기, 막대·원·수직선, 수 표현 패널, 같게 쪼개기·합치기가 한 화면에 있다

목적. 분수는 3~5학년에 걸쳐 누적되는 개념이지만 「1/3이 1/2보다 크다」, 「6/8은 3/4보다 크다」 같은 오개념이 가장 많이 굳어지는 영역이다. 분수교구함은 길이(막대)·넓이(원)·수(수직선) 세 표상과 분수 기호·소수·읽기를 하나의 상태로 묶어, 어느 쪽을 조작해도 나머지가 같은 순간에 바뀌는 가상 교구이다. 전자칠판 시연과 태블릿 개별 조작을 모두 겨냥했다.

- **분수 만들기**: 분자·분모 −/+ 버튼(분모 1~12, 분자 0~분모×2), 자주 쓰는 분수 칩 8 개, 막대·원의 칸을 직접 눌러 색칠(마지막 칸을 다시 누르면 하나 줄어듦), 키보드 ←/→·↑/↓.
- **세 모델 동시 표시**: 막대(길이)·원(넓이)·수직선(수)이 한 분수를 함께 보여 주고, 가분수이면 막대·원이 2 개로 늘고 수직선이 0~2 로 확장되며 대분수(7/4 = 1 3/4)를 병기한다. 보기 버튼으로 모델을 하나씩 끌 수 있다(최소 1 개 유지).
- **같게 쪼개기·합치기**: ×2·×3 을 누르면 모든 칸이 쪼개지는 분할선 애니메이션과 함께 색칠된 양이 그대로임을 보여 주고, 합치기(약분)로 되돌린다. 과정은 동치 사슬(3/4 = 6/8 = 3/4 …)로 쌓이며 항을 누르면 그 분수로 돌아간다.
- **수 표현 패널과 수업 옵션**: 소수(유한소수는 =, 순환소수는 ≈ 셋째 자리)와 한국어 읽기(4 분의 3)를 표시하고, 「예측 먼저」 수업을 위해 숨길 수 있다. 큰 글씨, 라이트/다크 테마, 마지막 상태 자동 저장.

단순화 과정. 처음 구현한 분수교구함(v1, 2,454줄)은 교구판·비교/통분·덧셈/뺄셈·탐구 카드 24장·장면 보관함·활동지의 6개 화면을 갖추고 3관점 검증을 1라운드에 통과했다. 그러나 「기능이 많으니 좀 더 심플하게」라는 사용자 피드백을 받아 수업에서 가장 자주 쓰는 교구판 하나만 남기기로 결정하고, 단

일 화면 스펙(기능 7개, 수용 기준 15개)을 새로 작성해 679줄로 다시 구현했다. v1 원본과 검증 기록은 `_dev/archive_fraction-kit-v1`에 보존했다.

카테고리 적합성과 교육적 가치. 수학 교과 분수 단원(3-1 → 3-2 → 4-2 → 5-1) 수업을 직접 돕는 교구이며, 학생 개인별 기록(학습지도)과 점수·보상(게임)은 두지 않는다. 실물 분수 막대가 한두 세트뿐이고 손으로 그린 막대·원은 비율이 부정확한 교실에서 「쪼개도 양은 같다」를 정확한 그림과 애니메이션으로 보여 줄 수 있고, 소수·읽기를 숨긴 채 예측하게 한 뒤 확인하는 수업에 바로 쓸 수 있다. 화면이 하나뿐이라 처음 보는 교사도 설명 없이 사용할 수 있다.

2.2 기억상자 — 02. 학습지도용 앱

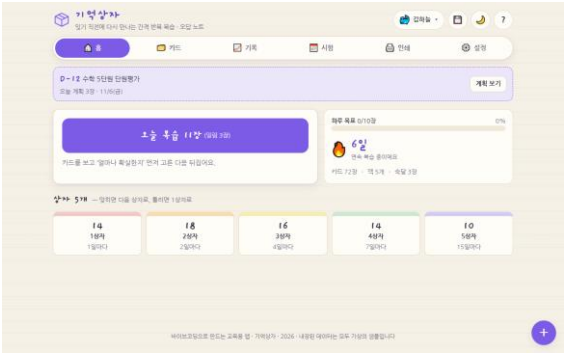


그림 2. 기억상자 첫 화면(홈) — 오늘 복습 큐, 하루 목표 진행 바, 상자 5개

목적. 초등학생은 받아쓰기 낱말·영어 단어·사회과학 용어·수학 오답을 시험 전날 몰아서 보고 금방 잊으며, 오답노트는 베껴 쓰는 데서 끝나기 쉽다. 기억상자는 카드를 라이트너 5단계 상자에 넣고 「오늘 복습할 카드」만 자동으로 골라 주는 간격 반복 복습·오답 노트 앱으로, 학생의 자기주도 복습을 지도하고 교사는 학급 오답을 집계한다.

- **라이트너 스케줄러**: 상자 1~5 간격 1·2·4·7·15 일, 5 상자에서 확실히 맞히면 30 일(숙달). 오늘 큐는 가장 많이 밀린 카드부터 나온다.
- **확신도 → 뒤집기 → 자기 채점**: 뒤집기 전에 모름/애매/확실을 고르게 하여 「아는 것과 모르는 것」을 구별하는 메타인지를 훈련하고, 세션 요약은 확신도×결과 4 분면과 코칭 한 줄을 보여 준다.
- **오답 원인 태그와 기록 대시보드**: 원인 5 종을 붙인 오답 카드가 도넛 차트로 실수 패턴을 드러내고, 12 주 잔디 달력·상자 분포·망각 예측선을 직접 그린 SVG 로 제공한다. 시험 D-Day 계획, 암기카드 양면·다시풀기 학습지·A5 주간 리포트 인쇄를 갖추었다.

- **교사 모드**: 학생들의 복습 요약 JSON 을 모아 단원×원인 표, 복습 포인트, 집중 지도 학생을 집계하고 CSV 로 내보낸다.

카테고리 적합성과 교육적 가치. 과목을 가리지 않는 학습 방법(간격 반복·인출 연습), 학습 습관(하루 목표·스트릭·잔디 달력), 자기 점검(4분면·원인 도넛·되새김 한 줄)을 지원하는 점에서 학습지도용 앱에 해당한다. 점수·레벨·랭킹·효과음을 두지 않아 교육용 게임과 구별되고, 생활기록부 같은 행정 산출물은 만들지 않는다. 「상자 옮기기」라는 한 가지 규칙으로 간격 반복을 아이 눈높이에 맞게 체화시키며, 교사는 학급 공통 오답 단원과 원인을 근거로 다음 수업의 복습 지점을 정할 수 있다.

2.3 자리뚝딱 — 03. 학교 업무 자동화 앱

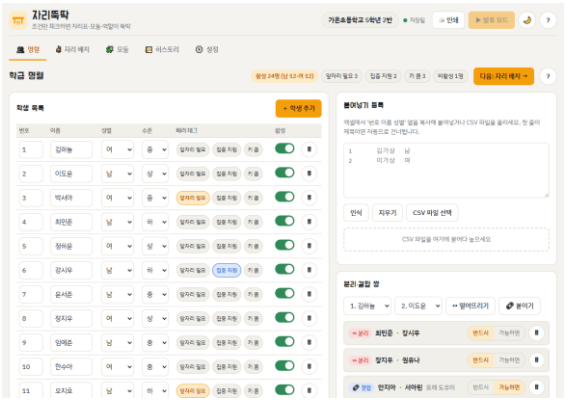


그림 3. 자리뚝딱 첫 화면(명렬 탭) — 샘플 학급 학생 표, 요약 칩, 붙여넣기 등록, 분리·결합 쌍

목적. 담임교사는 월 1회꼴로 자리를 바꾸고 수업마다 모둠을 다시 짜면서 「시력 나쁜 아이는 앞자리」, 「이 둘은 떨어뜨리기」, 「남녀 짝」, 「지난번 짝은 피하기」 같은 조건을 머릿속으로 동시에 맞춘다. 자리뚝딱은 명렬과 조건 체크만 입력받아 제약 만족 알고리즘으로 자리표·모둠표·역할표를 자동 생성하고, 공개·인쇄·이력 관리까지 이어 주는 담임 업무 자동화 앱이다.

- **자동 배치 엔진**: 조건 7 종(하드/소프트)을 패널티 함수로 평가하고 언덕 오르기(2,000 회·최대 30 회 재시작·800ms 상한)로 1 초 안에 좌석표를 만든다. 충족 리포트·위반 좌석 빨간 테두리·완화 제안을 같은 평가 함수에서 산출한다.
- **수동 조정과 모둠 편성**: Pointer Events 드래그 교환, 탭-탭 교환, 즉시 재검사, 자물쇠 핀 고정, 실행 취소 50 단계. 모둠은 성별·학습수준 균형과 최근 N 회 같은 모둠 회피를 반영하고, 역할은 경험이 가장 적은 학생에게 우선 배정한다.
- **발표 모드와 인쇄 4 종**: TV·전자칠판에서 카드가 0.7 초 간격으로 뒤집히며 공개되고, 게시용·교사용(180° 회전)·모둠 명단표·접이식 명패를 A4 로 인쇄한다.

- **공정성 대시보드**: 학생×학생 짝·모둠 히트맵, 앞/중/뒤 구역 누적 막대와 「편중 주의」 배지, 역할 경험 횟수 표.

카테고리 적합성과 업무적 가치. 자리 배치·모둠 편성·명패 제작은 교과 지도나 평가가 아니라 담임이 반복 수행하는 학급 운영 업무이다. 사람이 하던 판단(조건 동시 충족)을 알고리즘이 대신하고 뒤따르는 문서 작업(인쇄 4종)까지 자동화하므로 학교 업무 자동화 앱에 부합하며, 특별실 예약 같은 학교 단위 행정은 포함하지 않는다. 회당 30분 이상 걸리던 작업이 조건 체크와 확인 중심의 몇 분 작업으로 바뀌고, 발표 모드는 「컴퓨터가 공정하게 뽑았다」는 공개 절차를, 히트맵은 학부모 상담에서 공정성을 설명하는 근거를 제공한다.

2.4 특별실지기 — 04. 학교 행정 지원 앱

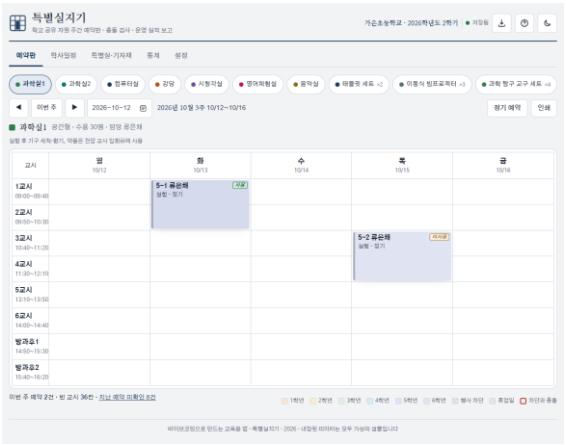


그림 4. 특별실지기 첫 화면(예약판) — 자원 칩, 주 내비게이션, 요일×교시 격자와 예약 블록

목적. 초등학교의 특별실·기자재 예약은 교무실 화이트보드와 종이 대장으로 이루어져 같은 교시에 두 학급이 겹치거나 운동회 날에도 예약이 잡히는 실수가 반복되고, 태블릿 세트처럼 수량이 있는 기자재는 남은 수량을 아무도 모른다. 특별실지기는 요일×교시 격자 위에서 예약을 드래그로 등록하고 저장 전에 충돌·휴업일·행사 차단·수량 부족을 자동 검사하며, 학기말 운영 실적과 학급별 이용 형평성까지 집계하는 학교 공유 자원 예약판이다.

- **주간 교시 격자 예약판**: 자원 칩 선택 후 빈 셀 클릭(1 교시) 또는 같은 요일 드래그(연속 교시)로 예약. 휴업일 열·행사 차단 빗금·오늘 열 강조.
- **교시 단위 충돌 검사와 대안 추천**: 하나의 `checkConflict` 함수가 공간형(수량 1)과 수량형(태블릿 세트 2 등) 자원을 같은 알고리즘으로 검사하고, 같은 날 다른 교시·같은 그룹 자원·다른 요일 대안 칩을 최대 3 개 제안한다.

- **학사일정 연동과 정기 예약**: 공휴일·재량휴업일·학교행사 등록이 곧바로 예약 차단으로 이어지고, 정기 예약은 미리보기(휴업일 제외·충돌 표시)를 거쳐 시리즈로 일괄 생성한다.
- **인쇄와 통계**: 결재란이 포함된 A4 가로 주간 이용 현황표와 A4 세로 운영 실적 보고서, 집계 CSV, 자원별 가동률·학급×자원 히트맵(「저조」 표시)·요일×교시 혼잡도.

카테고리 적합성과 업무적 가치. 학생 개인 데이터를 전혀 다루지 않고, 담당 부서가 맡는 자원 배정·조정·실적 보고라는 학교 단위 행정 업무를 대상으로 한다. 교무실 화이트보드를 그대로 디지털화한 문제 정의와 결재란이 포함된 공문서 성격의 산출물이 행정 지원 앱의 특성에 부합한다. 이중 예약과 행사일 예약 실수를 저장 단계에서 차단하고 종이표를 세어 엑셀로 옮기던 학기말 집계를 자동화하며, 형평성 히트맵은 학급 간 특별실 이용 격차를 드러내어 예약표를 「학습 기회 분석 도구」로 확장한다.

2.5 연산수호대 — 05. 교육용 게임 앱

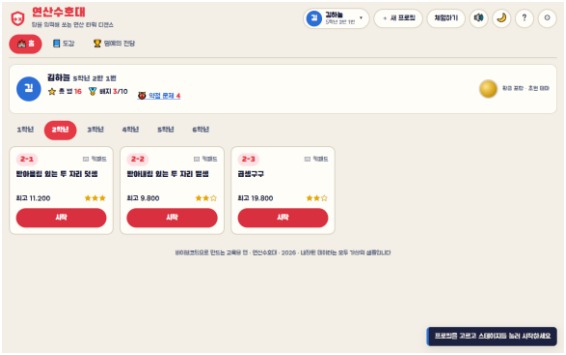


그림 5. 연산수호대 첫 화면(홈·스테이지 맵) — 프로필 요약, 학년 탭, 스테이지 카드

목적. 연산 반복 연습은 꼭 필요하지만 지루하다. 연산수호대는 1~6학년 교육과정의 연산 영역(덧셈·뺄셈·곱셈구구·나눗셈·분수·소수·비율)을 실시간 타워 디펜스 위에 올려 「성을 지키는 한 판」으로 바꾼다. 문제를 머리 위에 단 몬스터가 성으로 다가오고, 학생이 정답을 입력하면 포탑이 그 몬스터를 자동 조준해 격파한다.

- **Canvas 전투 루프와 규칙 기반 문제 생성기**: 960×540 논리 해상도, requestAnimationFrame(dt 상한 50ms), 포탑·포탄·파티클·화면 흔들림. 학년 탭 6 개 × 스테이지 19 개의 문제를 전부 규칙으로 즉석 생성하며 소수는 정수 스케일·분수는 기약분수로 계산해 부동소수점 오차가 없다.
- **게임 메커니즘**: 콤보 배율 ×1~×4, 성 HP 3, 10 웨이브 보스(방패 3 개), 필살기 게이지, 별 1~3 개, 적응형 난이도, 배지 10 종과 포탑 스킨·배경 테마 해금.

- **학습 엔진**: 오답 즉시 시각 모델 교정 카드(점 배열·세로셈·분수 막대 등), 같은 판의 「복수 몬스터」(점수 2 배), 다음 판의 약점 큐 20% 출제, 사실(fact)별 숙달 상자, 「다시 풀기」 복습 모드.
- **Web Audio 합성 효과음과 교실 운영**: 오디오 파일 없이 효과음을 합성하고, 프로필 최대 40 명·명예의 전당·선생님 설정(활성 스테이지·속도·시간 제한·PIN)을 제공한다.

카테고리 적합성과 교육적 가치. 문제 풀이에 점수를 붙인 퀴즈가 아니라 실시간으로 몬스터가 전진하고 포탑이 발사하며 파티클·효과음이 반응하는 게임 루프를 갖추었고, 콤보·HP·웨이브·보스·별·배지·해금 같은 게임 고유의 동기 장치가 학습 내용과 맞물려 있다. 교사용 진단 대시보드나 개념 설명 화면은 두지 않아 다른 카테고리와 겹치지 않도록 범위를 한정했다. 틀린 문제가 교정 카드 → 복수 몬스터 → 약점 큐로 자동 반복되므로 교사가 따로 문제를 골라 줄 필요가 줄고, 성장 랭킹과 약점 문제 CSV는 보충 지도 대상 선정과 학부모 상담 자료로 쓸 수 있다.

3. 공통 설계와 기술 구조

다섯 앱은 서로 독립된 폴더이지만 하나의 포트폴리오처럼 보이고 동작하도록 공통 구현 규약(`_dev/CONVENTIONS.md`)을 먼저 정하고, 모든 빌더 에이전트에게 「규약을 스펙보다 우선한다」고 지시했다.

순수 HTML/CSS/JavaScript를 선택한 이유. 학교 PC·태블릿·전자칠판은 소프트웨어 설치와 네트워크 제약이 크다. 빌드 도구·서버·로그인 없이 `index.html` 더블클릭(`file://`)으로 열려야 교사가 바로 쓸 수 있고 심사자도 압축을 풀어 즉시 실행할 수 있다. CDN 스크립트·외부 API 키를 쓰지 않으므로 인터넷이 끊긴 교실에서도 동작하고 서비스 종료나 요금 문제가 없으며, 차트·드래그·애니메이션·달력·사운드는 SVG/Canvas/DOM/Web Audio로 직접 구현했다. 모든 데이터는 브라우저 localStorage에만 저장되고 JSON/CSV 파일로 입출력하므로 학생 정보가 외부 서버로 전송되지 않는다. 단일 페이지·단일 폴더 구조라 정적 점검과 브라우저 자동화(Playwright)로 다섯 앱을 같은 방식으로 검증할 수 있었다.

영역	공통 규약
파일 구조	`index.html` + `css/style.css` + `js/data.js`(샘플·상수) + `js/app.js`(길어지면 engine/ui/game 등으로 분리) + `README.md`(10절 템플릿). 폴더 밖 파일 참조 금지

영역	공통 규약
실행 환경	ES 모듈·`fetch` 로컬 파일 읽기 금지(file:// CORS), 외부 라이브러리 금지, Google Fonts만 허용(시스템 글꼴 대체 스택 필수), `alert/confirm/prompt` 금지(인앱 모달·토스트), 콘솔 오류 0건
테마·접근성	CSS 토큰 3블록(라이트 / OS 다크 / 수동 다크), 모든 색은 토큰으로만, 글자 대비 4.5:1 이상, safe-area 패딩, 400px 폭 가로 스크롤 금지, 터치 타깃 44px 이상, `:focus-visible`, `aria-label`, `prefers-reduced-motion` 대응, 첫 화면은 샘플이 채워진 작업 상태
데이터·저장	localStorage try/catch 래퍼(실패 시 메모리 모드 + 1회 안내), 저장 데이터에 `version`, JSON 전체 백업·CSV(UTF-8 BOM) 입출력, 가져오기 구조 검증, `innerHTML`에 넣는 입력은 `escapeHtml()` 필수
UI·코드·인쇄	`showToast`, `showModal → Promise<boolean>`(ESC·배경 클릭 닫기, 포커스 복귀), 파괴적 동작은 모달 확인, 상태 → 렌더 단방향, `data-action` 이벤트 위임, 앱당 1,200~2,500줄, 테스트용 `window.__app` API, 인쇄가 핵심인 앱만 `window.print()` + `@media print`

공통 샘플 데이터와 시각 아이덴티티. 다섯 앱은 가상의 가온초등학교 5학년 2반(담임 이가람, 학생 24명)을 공통 샘플로 내장하고 앱 성격에 맞게 확장했다(자리뚝딱은 전출 시연용 25번 비활성 학생, 특별실지기는 18개 학급·23명 교사·약 220건 예약, 연산수호대는 24명 중 6명에게만 게임 기록). 첫 실행 시 저장 데이터가 없으면 샘플이 자동 주입되고 「샘플 데이터로 초기화」로 복원할 수 있다. 액센트 색은 분수교구함 #3B82F6(교구·칠판), 기억상자 #7C5CE6(크림색 공책·손글씨), 자리뚝딱 #F59E0B(학급 게시판), 특별실지기 #1E3A5F(공문서풍 네이비·명조), 연산수호대 #E63946(게임 HUD)으로 앱마다 다르게 두어 포트폴리오 안에서 구별되게 했다.

4. 바이브코딩 개발 과정

개발은 Windows 10 PC에서 Claude Code를 에이전트로, Chrome/Edge를 실행 환경으로 사용했다. 모든 코드·테스트·문서 초안은 AI가 작성하고 개발자는 지시·검토·결정을 맡았다. 형상 관리는 Git과 GitHub 공개 저장소로 하고 GitHub Pages로 배포했다.

4.1 전체 절차와 각 단계의 지시 요지

단계	수행 내용	AI에게 준 지시의 요지
① 요구사항 정리	5개 카테고리·제출물 구성 확인	「5개 카테고리 앱을 폴더별로 개발. 제출물로 개요·설명서, 소스 압축, 구현 사이트 포함」
② 기술 제약 결정	순수 클라이언트, file:// 실행, localStorage, 한국어 UI, 반응형, 샘플 내장 (개발자 판단)	제약을 문장으로 명시. 「file://에서 동작」을 「ES 모듈·fetch 금지」로 구체화
③ 컨셉 제안	카테고리당 3관점 × 2개, 총 30개	「당신은 15년 경력 초등 담임교사(또는 교육공학 연구자 / 시니어 프론트엔드 개발자)입니다. 카테고리 N에 맞는 앱 컨셉 2개를 기술 제약 안에서, 한국 초등 현장의 용어·절차를 반영해 제안…」
④ 심사·스펙 확정	카테고리별 1개 선정, 구현 스펙 작성, 5개 스펙 정합성 검토	「6개 컨셉을 100점 기준(교육적 가치 25·현장 실용성 25·구현 가능성 20·차별성 20·적합성 10)으로 심사해 1개를 확정하고, 추가 질문 없이 구현할 수 있도록 기능(must 8개 이하)·화면·데이터 모델·알고리즘·샘플 데이터·엣지 케이스·수용 기준(테스트 가능한 문장)을 작성」
⑤ 공통 규약	CONVENTIONS.md 초안 (AI) → 확정(개발자)	파일 구조, file:// 보장, 테마 토큰, alert 금지, 저장 래퍼, 접근성, README 템플릿, 자가 점검 체크리스트
⑥ 구현	앱당 빌더 1명, 1,585~2,454줄(분수교구함 v1 기준)	「공통 규약을 먼저 읽고 규약을 스펙보다 우선. must 전부 구현, 샘플 내장, alert 금지, 라이트/다크 모두 디자인, 완료 전 lint 0건·스모크 0건·수용 기준 확인, README 템플릿 채우기」
⑦ 3관점 검증	앱당 3명 병렬, 소스 수정 금지	「목표는 결함을 찾는 것(통과시키는 것이 아님). 빌더의 테스트를 믿지 말고 독립 모듈을 작성. 입력 검증·XSS 문자열·새로고침 복원·가져오기 오류·400px·다크 모드까지」

단계	수행 내용	AI에게 준 지시의 요지
⑧ 수정·재검증	최대 3라운드, critical/major 0건이면 종료	수정:「critical·major는 반드시 수정, 지적이 틀렸다면 재현해 근거를 적고 not_fixed로. 기존 테스트 전부 재실행. 새 기능 추가 금지」/ 재검증:「수정 보고서를 믿지 말고 소스·브라우저로 직접 확인. 회귀 탐지」
⑨ 문서화·패키징	앱별 설명서, 과제 보고서, zip, GitHub Pages	README·스펙·검증 결과 JSON을 근거로만 서술하고 수치는 결과 파일의 값만 사용
⑩ 피드백 반영	분수교구함 단순화(6개 화면 → 교구판 1개), 679줄로 재구현	「분수교구함은 기능이 많으니 좀 더 심플하게」→ 범위를 묻는 질문에「교구판 하나만」으로 답해 범위 확정. 원본은 보관하고 새 스펙·브리프로 재구현
⑪ 최종 재점검	5개 앱 lint·스모크·기존 테스트 전체 재실행, 남은 오류 수정	「전체적으로 오류가 있는지 확인해주고 고쳐줘」→ 실패 항목을 실제 결함과 테스트 기대값 차이로 나눈 뒤 실제 결함만 수정하고 다시 실행

4.2 멀티 에이전트 워크플로

한 명의 AI에게 모든 일을 맡기는 대신 역할과 관점이 다른 여러 에이전트를 단계별로 배치하고 산출물을 파일(JSON·Markdown·스크린샷)로 넘겨 주었다. 각 에이전트는 자기 역할의 브리프(`_dev/briefs/<앱>.<단계>.md`)만 읽고 작업한다.

1. **관점별 제안·심사**: 카테고리마다「현장 교사 실용성」,「학습 효과·학생 참여」,「기술 완성도·시연 임팩트」관점의 에이전트가 각 2 개 컨셉을 제안하고, 심사 에이전트가 5 개 기준으로 채점해 1 개를 선정(필요 시 두 안을 합침)한 뒤 구현 스펙을 작성했다. 정합성 검토 에이전트가 5 개 스펙의 카테고리 경계 중복, 기술 다양성, 이름 톤, 공통 샘플 데이터, 시각 아이덴티티를 통일했다.
2. **구현(빌더)**: 앱당 1 명이 스펙과 규약을 읽고 앱 전체를 작성한 뒤 lint·스모크·수용 기준 자가 점검을 수행하고 빌드 보고서를 남겼다.
3. **3 관점 검증(병렬, 소스 수정 금지)**: ① 기능 테스트 — 독립 작성한 Playwright 모듈로 수용 기준과 공통 시나리오 실행, ② 코드 리뷰 — 논리·상태/렌더·데이터·XSS·규약·스펙 누락을 소스 정독으로 점검하고 결함은 재현 모듈로 확인, ③ UX·시각 리뷰 —

데스크톱·태블릿·모바일·다크·인쇄 스크린샷 수십~수백 장을 실제로 보고 판단. 결함은 critical/major/minor 로 기록했다.

4. **수정 라운드**: 수정 에이전트가 결함을 고치고 기존 테스트를 전부 재실행한 뒤, 독립 재검증 에이전트가 수정 주장을 믿지 않고 직접 확인(회귀 탐지)했다. critical/major 잔여 0 건이면 종료, 아니면 다음 라운드로 넘어가며 최대 3 라운드로 제한했다.

> 재검증 에이전트는 세 앱에서 수정 에이전트가 「통과」라고 보고한 뒤에도 새 major 결함을 찾아냈다. 기억상자에서는 XSS 정화가 일부 필드에만 적용된 부분 수정, 자리뚝딱에서는 핀 고정 학생의 옆자리가 「다시 섞기」마다 비워지는 1라운드 수정의 회귀, 연산수호대에서는 보스전에서 정답 발사 직후 필살기를 쓰면 조준이 고착되는 소프트락이었다. 세 건은 모두 2라운드에서 수정되고 재검증을 통과했다.

4.3 자동화 검증 도구

검증을 AI의 「통과했다」는 말에 의존하지 않도록 AI에게 재사용 가능한 점검 도구를 먼저 만들게 하고, 모든 앱·모든 라운드에서 같은 도구를 돌렸다(`_dev/tools/`). 모든 앱은 테스트 용이성을 위해 `window.__app = { version, getState(), actions }`를 노출하며, 테스트 모듈은 화면을 실제로 클릭·입력하되 상태 확인은 이 읽기 전용 스냅샷으로 하여 DOM 구조 변경에 덜 흔들리는 단언을 작성할 수 있었다.

도구	역할	점검 항목
`lint-app.js`	정적 점검	`alert/confirm/prompt`, ES 모듈·외부 리소스, 테마 토큰 3블록, safe-area, focus-visible, reduced-motion 등 규약 위반 탐지, 코드 줄 수 집계
`pw-check.js`	스모크 점검 (Playwright + Edge)	콘솔 오류·페이지 예외·실패 요청·dialog 0건, 400px 가로 스크롤 여부, 데스크톱 라이트/다크·모바일 3개 뷰 스크린샷
`pw-run.js`	인터랙티브 테스트 러너	검증 에이전트가 작성한 모듈을 실행해 클릭·입력·새로고침·파일 가져오기로 수용 기준을 단언(expect)하고 스크린샷 촬영. 최종 라운드 기준 앱당 11~24개 모듈
`collect-qa.py`, `summarize-findings.py`	결과 집계	빌드·검증·수정·재검증 보고서를 앱별 JSON(`_dev/qa.json`)으로 모아 수치를 한 곳에서 참조

도구	역할	점검 항목
`make-docx.js`, `docx2pdf.ps1`, `package.py`	문서·패키징	콘텐츠 JSON → docx, PDF 변환, 소스 zip 생성

4.4 반복 횟수와 결함 수

아래 수치는 빌드·검증 결과 JSON(`_dev/qa.json`)의 값이다. 결함 수는 기능 테스트·코드 리뷰·UX 리뷰 세 보고서의 합계이므로 같은 결함이 둘 이상의 보고서에 기록된 중복을 포함한다.

항목	수치
제안된 컨셉	30개(5개 카테고리 × 3관점 × 2개) → 확정 스펙 5개
구현 기능	must 38 · should 24 · nice 9 = 71개 중 68개 구현, 3개 보류(모두 nice: 상자 간 드래그 조정, 1인 1역 주간 로테이션표, 예약 블록 드래그 이동)
수용 기준 / 코드 줄 수	1차 개발: 수용 기준 78개(17 · 15 · 15 · 13 · 18) 전부 통과, 10,012줄(README 제외). 현재: 분수교구함 단순화판 679줄(수용 기준 15개)로 교체되어 총 8,237줄(679 · 1,585 · 2,118 · 2,069 · 1,786)
검증 보고서·발견 결함	15건(앱당 3건)에서 273건 = critical 13 · major 71 · minor 189(앱별 57 · 61 · 45 · 43 · 67)
수정 라운드	분수교구함·특별실지기 1라운드 통과, 기억상자·자리뚝딱·연산수호대 2라운드 통과(허용 최대 3라운드). 재검증에서 새로 발견된 major 3건(앱당 1건)은 2라운드에서 수정
수정·보류	1라운드 265건 + 2라운드 8건 수정. 보류 8건은 모두 minor(스펙 갭·새 기능 추가에 해당)로 각 README「제한 사항」에 명시
단순화·최종 재점검	분수교구함 단순화판 독립 검증에서 결함 9건 수정, 최종 재점검에서 특별실지기 minor 2건(1라운드 재검증 때 남긴 차트 넘침·모달 표 가로 스크롤) 수정

4.5 프롬프트 작성에서 배운 점

- **제약과 수용 기준은 테스트 가능한 문장으로.** 「file://에서 동작」이라고만 쓰면 결과가 들쭉날쭉했지만 「ES 모듈·fetch 금지」로 구체화하자 편차가 크게 줄었다. 수용 기준도 「새로고침 후 마지막 탭·분수·단계가 복원된다」처럼 써야 자동 테스트로 옮길 수 있었다.
- **만드는 사람과 검증하는 사람을 분리한다.** 빌더의 「수용 기준 전부 통과」 보고는 DOM 순서만 확인한 경우가 있었다. 독립 검증에서 교사용 자리표 180° 회전 시 격자 붕괴, 가져오기 JSON 의 깊은 검증 부재, 가져온 문자열의 XSS 등 빌더가 놓친 결함이 드러났다.
- **UX 리뷰는 스크린샷을 실제로 보게 해야 한다.** 글자 크기 6~9px, 대비 2.2:1, 모바일 메뉴 잘림 같은 문제는 코드만 읽어서는 발견되지 않았다.
- **수정 에이전트에게는 「새 기능 추가 금지」와 「지적이 틀렸다면 근거를 적고 not_fixed 로」를 함께 준다.** 이 두 문장이 있어야 수정 범위가 통제되고 보류 사유가 README 에 남는다.
- **교육 현장 맥락은 명시해야 반영된다.** 생활기록부·특별실·학사일정·전자칠판 시연 같은 맥락은 프롬프트에 적은 만큼만 결과에 들어갔다.
- **중간 결과는 파일로 남기고 커밋한다.** 세션 중단(사용량 제한)이 잦아 모든 산출물을 JSON·스크린샷·Markdown 으로 저장하고 git 에 커밋하는 습관이 재개 비용을 줄였다. 임시 폴더에 둔 테스트 모듈이 세션 교체 시 사라진 뒤로는 프로젝트 `_dev/` 아래에 보관했다.
- **기능의 수가 곧 교구의 가치는 아니다.** 검증을 통과한 6 개 화면의 분수교구함도 사용자에게는 「기능이 많다」로 받아들여졌다. 바이브코딩은 다시 만드는 비용이 낮으므로, 기능을 덧붙이기보다 핵심 장면 하나를 남기고 다시 만드는 결정이 더 쉬웠다. 범위를 바꿀 때는 AI 가 임의로 줄이지 않도록 「어디까지 남길지」를 먼저 질문해 확정했다.

5. 품질 검증 결과

1차 개발에서 다섯 앱 모두 최종 재검증 critical/major 잔여 0건, 수용 기준 전부 통과, lint 0건, 스모크 점검(콘솔 오류·페이지 예외·dialog·400px 가로 스크롤) 0건으로 **pass** 판정을 받았다. 아래 표의 분수교구함 행은 단순화 이전 v1의 검증 기록이며, 단순화판의 검증과 2026-10-28 최종 재점검 결과는 표 아래에 따로 적었다.

앱	결함 발견 (기능/코드/UX = 합계)	critical / major / minor	수정 라운드	재검증 결과	수용 기준	코드 줄 수
분수교구함 (v1)	11 / 24 / 22 = 57	2 / 16 / 39	1	1R pass	17/17	2,454
기억상자	16 / 20 / 25 = 61	6 / 13 / 42	2	1R fail(신규 major 1) → 2R pass	15/15	1,585
자리뚝딱	9 / 15 / 21 = 45	3 / 17 / 25	2	1R fail(회귀 major 1) → 2R pass	15/15	2,118
특별실지기	6 / 17 / 20 = 43	1 / 10 / 32	1	1R pass	13/13	2,069
연산수호대	14 / 26 / 27 = 67	1 / 15 / 51	2	1R fail(신규 major 1) → 2R pass	18/18	1,786
합계	56 / 102 / 115 = 273	13 / 71 / 189	8	5개 앱 모두 pass	78/78	10,012(v1 기준)

최종 라운드에서 재실행한 자동 테스트의 규모는 분수교구함 16개 모듈 686 passed, 기억상자 13개 모듈 649 passed(재검증 53·XSS 회귀 40 별도), 자리뚝딱 752 passed(엔진 node 단위 테스트 포함), 특별실지기 13개 모듈 542 passed, 연산수호대 24개 모듈 703 passed(node 생성기 검증 113 별도)이며 모두 0 failed·pageErrors 0·consoleErrors 0·dialogs 0이었다. 연산수호대의 병렬 실행 타이밍 플레이크 1건은 단독 재실행에서 통과했다.

분수교구함 단순화판 검증. 빌더 자가 점검(lint 0건, 스모크 0건, 수용 테스트 42개 단언 통과) 뒤 독립 검증 테스트 4개 모듈(핵심 기능·설정·반응형·코드 결함 재현)을 실행해 실제 결함 9건을 찾아 수정했다: ① 400px·큰 글씨에서 대분수/읽기 문자열로 생기는 가로 스크롤, ② 좁은 화면에서 분모 12 수직선 라벨 겹침(실제 표시 폭 기준으로 라벨 간격 계산), ③ 저장된 동치 사슬 항의 분자 범위 미검증, ④ 사슬에서 현재 분수가 두 번 강조됨, ⑤ `setShow(모델, false)` 반복 호출 시 다시 켜짐, ⑥ 잘못된 테마 저장값 적용, ⑦ 모달 안에서 시작한 드래그를 배경에서 놓으면 모달이 닫힘, ⑧ 400px에서 분모 9 이상일 때 원 조각 번호가 분할선을 넘음, ⑨ 사슬 앞쪽 항으로 돌아간 뒤 쪼개면 같은 항이 뒤에 다시 붙음. 남은 실패 항목은 테스트의 문자열 기대값 차이(예: 「소수 = 0.75」와 실제 표시 「소수」 + 「= 0.75」)이거나 의도한 설계(「처음으로」는 사슬의 첫 분수로 되돌림, 화살표 키는 분수 조작 단축키, 400px에서

분모 12 막대 칸 폭 28px는 물리적 한계)여서 수정하지 않았다.

2026-10-28 최종 재점검. 제출 직전에 다섯 앱과 허브 페이지 전체에 정적 점검(lint)과 스모크 점검을 다시 돌려 모두 0건임을 확인하고, 그동안 작성된 기존 테스트 모듈을 전부 재실행했다. 결과는 기억상자 10개 모듈 605 passed / 0 failed(확신도 버튼 순서를 「모르겠어요 → 애매해요 → 확실해요」로 바꾼 뒤에도 통과), 자리뚝딱 9개 모듈 560 passed / 0 failed, 특별실지기 13개 모듈 633 passed / 0 failed, 연산수호대 19개 모듈 607 passed / 2 failed였다. 특별실지기는 첫 실행에서 1라운드 재검증 때 minor로 남겨 두었던 두 건(데스크톱 통계의 「학년별 이용 횟수」 차트가 카드 밖으로 넘침, 400px에서 미확인 예약 모달을 열면 표가 오른쪽으로 스크롤된 채 시작)이 다시 드러나 이번에 수정했고, 수정 후 해당 모듈 포함 전부 통과했다. 연산수호대의 실패 2건은 보스 방패의 약점 문제 출제를 확인하는 테스트였다. 하나는 단독 재실행에서 통과한 타이밍 문제였고, 다른 하나는 탐침 실행으로 확인한 결과 시뮬레이션 학생이 6웨이브까지 약점 사실을 모두 두 번씩 맞혀 보스 등장 시점에 약점(상자 2 미만)이 남아 있지 않았기 때문이었다. 이는 앱의 정의대로 동작한 것이어서 결함으로 보지 않았다. 모든 실행에서 페이지 예외·콘솔 오류·브라우저 대화상자는 0건이었다.

대표적인 critical 결함은 세 앱(분수교구함·기억상자·연산수호대)에서 발견된 **가져온 JSON 문자열의 XSS**(id·라벨·프로필 필드가 이스케이프 없이 `innerHTML`에 삽입)와, 역시 세 앱(기억상자·자리뚝딱·특별실지기)에서 발견된 **형식 검증이 얕은 백업 복원으로 기존 데이터가 지워지거나 앱이 뜨지 않는 문제**였다. 수정 라운드에서 가져오기 시점의 구조·형식 검증(id 정규식, 허용 목록, 숫자 강제)과 렌더 시점의 이스케이프를 모두 적용하고 전용 회귀 테스트 모듈을 추가했다. 보류한 8건은 모두 minor로, 스펙과 구현 사이의 작은 차이(칠판 모드의 세로 아이콘 바, 좁은 화면 탭 「더보기」 묶음 등) 또는 「새 기능 추가 금지」 원칙에 해당하며 각 README의 「제한 사항 및 향후 개선」에 사유와 함께 기록했다.

6. 활용 방안과 기대 효과

앱	활용 장면	기대 효과
분수교구함	3~5학년 분수 단원 전자칠판 시연(큰 글씨), 「예측 먼저」 수업(소수·읽기 숨김), 태블릿 개별 탐구, 동치분수 도입	「쪼개도 양은 같다」를 정확한 그림과 애니메이션으로 보여 주고, 화면 하나로 설명 없이 바로 수업에 투입
기억상자	아침 활동 10장 복습 루틴, 받아쓰기·영어 단어 덱 배부, 단원평가 오답 처방, A5 주간 리포트로 가정 연계	「베껴 쓰는 오답노트」를 인출 연습으로 바꾸고, 교사는 학급 공통 오답 원인을 근거로 복습 지점을 정함

앱	활용 장면	기대 효과
자리뚝딱	월초 자리 바꾸기, 실험·프로젝트 모둠 편성, 학기 말 공정성 점검과 학부모 상담 자료	30분 이상 걸리던 배치 업무가 몇 분으로 줄고, 공개 절차와 이력 히트맵으로 공정성을 설명
특별실지기	교무실 공용 PC 예약판, 학기 초 정기 예약, 운동회 기간 강당 차단, 학기말 운영 실적 보고	이중 예약·행사 날 예약 실수를 저장 단계에서 차단하고 종이표 집계를 자동화, 학급 간 이용 격차를 드러냄
연산수호대	2학년 곱셈구구 아침 5분 루틴, 기초학력 보충 지도(약점 문제 CSV·느린 속도), 전자칠판 도감 히트맵 공유	연산 연습의 지루함을 게임 루프로 바꾸고, 틀린 문제가 자동 반복되어 교사의 문제 선별 부담을 줄임

다섯 앱의 공통 기대 효과는 **설치·계정·네트워크 없이 바로 쓸 수 있다는 점**이다. 학교 현장에서 새 도구의 도입을 막는 벽은 기능 부족보다 설치 권한·망 분리·계정 발급 절차이며, `index.html` 더블클릭과 JSON 파일 교환만으로 동작하는 구조는 이 벽을 우회한다. 또한 교사가 바이브코딩으로 자기 교실의 문제에 맞춘 도구를 직접 만들 수 있다는 가능성을 과제 수행 과정 자체가 보여 준다.

7. 한계와 향후 과제

- **단일 기기·단일 브라우저 저장**: 모든 데이터가 localStorage 에만 있어 기기 간 이동은 JSON 백업으로만 가능하고 브라우저 데이터 삭제 시 함께 지워진다. 여러 교사가 함께 쓰는 특별실지기는 공용 PC 1 대 운영 또는 JSON 공유·병합을 전제하며, 로그인·권한·알림·나이스 연동은 범위 밖이다.
- **현장 검증 미실시**: 모든 검증은 자동 테스트와 AI·개발자의 리뷰로 이루어졌으며 실제 교사·학생이 교실에서 사용한 결과는 아직 없다. 사용성·학습 효과에 관한 서술은 설계 의도이지 측정 결과가 아니다.
- **AI 검증의 한계**: 3 관점 검증과 재검증을 거쳤음에도 재검증 단계에서 세 건의 major 결함이 추가로 발견되었다. 라운드를 거듭할수록 발견 수가 줄었지만 발견되지 않은 결함이 없다고 단정할 수는 없다.
- **기능 범위**: 분수교구함은 단순화 과정에서 비교·통분, 덧셈·뺄셈, 탐구 카드, 장면 보관함, 활동지를 제외했다(v1 에 보존). 분수의 곱셈·나눗셈, 카드별 난이도 계수(SM-2), 자유 좌표 교실

배치, 분 단위 대관, 멀티플레이 등도 의도적으로 제외했다. 보류한 nice 기능과 minor 항목은 각 앱 설명서에 기록되어 있다.

- **세션 중단 비용**: AI 사용량 제한으로 세션이 자주 끊겨 재개 비용이 발생했다. 중간 산출물을 파일로 남기는 습관으로 완화했지만 장시간 작업 계획에는 여전히 변수이다.

향후에는 (1) 실제 학급에서 한 단원 분량의 시범 적용과 교사 피드백 수집, (2) 보류 기능과 각 README의「향후 개선」항목 반영, (3) 공유 드라이브 JSON 병합 같은 파일 기반 공유의 사용성 개선, (4) 이번에 정립한「공통 규약 + 멀티 에이전트 검증」절차를 다른 교육용 앱 개발에 재사용하는 것을 과제로 삼는다.

8. 소감

이 과제를 시작할 때 가장 걱정한 것은「AI가 만든 앱을 내가 책임질 수 있는가」였다. 코드를 직접 쓰지 않고 다섯 개의 앱을 만든다는 것은, 바꾸어 말하면 내가 한 줄씩 읽지 않은 수천 줄의 코드를 제출한다는 뜻이기 때문이다. 그래서 개발 자체보다 검증 구조를 만드는 데 더 많은 지시를 썼다. 만든 에이전트와 검증하는 에이전트를 나누고, 검증하는 에이전트에게「통과시키는 것이 아니라 결함을 찾는 것이 목표」라고 적어 주고, 수정 뒤에는 다시 다른 에이전트가 의심하게 했다. 그 결과 273건의 결함 기록이 남았다. 처음에는 이 숫자가 부끄러웠지만 지금은 이 숫자가 있어서 제출할 수 있다고 생각한다. 빌더가「수용 기준 전부 통과」라고 보고한 앱에서 가져온 파일로 스크립트가 실행되는 결함이 나왔고, 수정 에이전트가 고쳤다고 한 뒤에도 재검증에서 회귀가 잡혔다. AI의「됐습니다」는 확인의 시작이지 끝이 아니라는 것을 숫자로 배웠다.

교사의 입장에서 가장 크게 느낀 것은 바이브코딩에서 가장 중요한 기술이 코딩이 아니라 **요구를 문장으로 정확히 쓰는 능력**이라는 점이다.「file://에서 동작해야 한다」와「ES 모듈과 fetch를 쓰지 말라」는 같은 뜻이지만 결과는 달랐고,「생활기록부」,「특별실」,「전자칠판 시연」처럼 현장의 말을 적은 만큼만 앱에 현장이 들어왔다. 이것은 교사가 학생에게 과제를 제시할 때 성취 기준을 구체적으로 쓰는 일과 다르지 않았다. 다섯 앱은 아직 실제 교실에서 쓰이지 않았고 세션이 끊길 때마다 흐름을 다시 잡는 데 시간이 들었지만, 설치 없이 열리는 앱 다섯 개와 그것을 어떻게 만들고 어떻게 의심했는지에 관한 기록을 함께 제출할 수 있어서 과제의 목표에는 닿았다고 생각한다. 또 하나 배운 것은「많이 만드는 것」보다「덜어 내는 것」이 어렵다는 점이다. 검증까지 통과한 분수교구함의 여섯 화면을 교구판 하나로 줄이자 오히려 수업 장면이 선명해졌다. 다음에는 이 절차를 그대로 가져와 내 교실의 문제 하나를 더 작게, 더 빨리 만들어 보고 싶다.

부록 A. 제출물 목록과 폴더 구조

- **전체**: 과제 보고서 상세본·요약본(본 문서와 별도 요약 문서, 각 docx/pdf) — `과제제출물/`; 허브 페이지와 저장소 README — `index.html`, `README.md`.
- **앱별 ×5**: 개요 및 설명서(docx/pdf)와 스크린샷 — `<앱 폴더>/docs/`(사본은 `과제제출물/`); 소스 압축(zip) — `과제제출물/`; 실행 가능한 소스 — `<앱 폴더>/`.
- **개발 과정**: 최종 스펙, 공통 규약, 브리프, 빌드·검증 결과 JSON, 검증 도구, 테스트 모듈 — `_dev/`.

```
ginue-2026_02_ai_course-homework/
├ index.html                      허브 페이지(다섯 앱 안내판)
├ README.md                       저장소 안내
├ 01_교과교육지원_분수교구함/       index.html, css/, js/(1개), README.md, docs/(설명서·스크린샷)
├ 02_학습지도_기억상자/            index.html, css/, js/(3개), README.md, docs/
├ 03_업무자동화_자리뚝딱/          index.html, css/, js/(4개), README.md, docs/
├ 04_행정지원_특별실지기/          index.html, css/, js/(4개), README.md, docs/
├ 05_교육용게임_연산수호대/        index.html, css/, js/(6개), README.md, docs/
├ 과제제출물/                     과제 보고서(상세·요약) · 앱별 개요및설명서(docx/pdf) · 앱별 소스 zip
└ _dev/                           CONVENTIONS.md, final_specs.json, qa.json, process-summary.md,
                                  briefs/, tests/, tools/(lint-app.js, pw-check.js, pw-run.js,
make-docx.js …)
```

부록 B. 실행 방법

1. 소스 압축을 풀거나 저장소를 내려받은 뒤 최상위 폴더의 `index.html`(허브 페이지)을 더블클릭하면 다섯 앱의 안내판이 열린다. 각 앱 폴더의 `index.html`을 직접 열어도 된다.
2. 권장 브라우저는 최신 Chrome 또는 Edge 이다. 설치·서버·로그인·인터넷 연결이 필요하지 않다(인터넷이 없으면 Google Fonts 대신 시스템 글꼴로 표시되며 기능은 동일하다). 로컬 서버로 열어도 동일하게 동작한다(예: `python -m http.server 8000`).
3. 첫 실행 시 가상의 샘플 데이터(가온초등학교 5 학년 2 반)가 들어 있다. 데이터는 브라우저 localStorage 에만 저장되므로 다른 PC 로 옮기려면 각 앱의 **JSON 내보내기/가져오기**를 사용하고, 「샘플 데이터로 초기화」로 처음 상태로 되돌릴 수 있다.

4. 전자칠판·태블릿에서는 터치로 조작할 수 있으며, 분수교구함의 큰 글씨, 자리뚝딱의 발표 모드, 연산수호대의 전체화면·큰 글씨 옵션을 함께 사용한다. 공용 PC 에서 사용한 뒤에는 각 앱 설정의 「전체 데이터 삭제」 또는 「샘플 데이터로 초기화」를 권장한다.

부록 C. 온라인 실행 링크

구분	주소
GitHub Pages 허브(다섯 앱 안내판)	https://changsubkwak.github.io/ginue-2026_02_ai_course-homework/
GitHub 소스 저장소	https://github.com/ChangSubKwak/ginue-2026_02_ai_course-homework
01 분수교구함	https://changsubkwak.github.io/ginue-2026_02_ai_course-homework/01_교과교육지원_분수교구함/
02 기억상자	https://changsubkwak.github.io/ginue-2026_02_ai_course-homework/02_학습지도_기억상자/
03 자리뚝딱	https://changsubkwak.github.io/ginue-2026_02_ai_course-homework/03_업무자동화_자리뚝딱/
04 특별실지기	https://changsubkwak.github.io/ginue-2026_02_ai_course-homework/04_행정지원_특별실지기/
05 연산수호대	https://changsubkwak.github.io/ginue-2026_02_ai_course-homework/05_교육용게임_연산수호대/

온라인 실행 페이지와 내려받은 소스는 동일한 코드이다. 온라인에서는 브라우저별 localStorage에 데이터가 저장되므로 같은 PC라도 브라우저가 다르면 데이터가 공유되지 않는다.